Lab Exercise 15- Creating ConfigMaps in Kubernetes

Name-Misha

SAP ID- 500119679

Batch-2

In this lab exercise, you will learn how to create ConfigMaps in Kubernetes using different methods: from literal values, from a file, from an environment file, and from multiple files. ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable.

Step 1: Set Up Kubernetes Cluster

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

Step 2: Create a ConfigMap from Literal Values

Create a ConfigMap named example-configmap with literal values:

```
kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
```

```
PS D:\kubs-misha> kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
configmap/example-configmap created
```

Verify the ConfigMap:

```
kubectl get configmap example-configmap -o yaml
```

```
configmap/example-configmap created
PS D:\kubs-misha> kubectl get configmap example-configmap -o yaml
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-17T07:30:39Z"
  name: example-configmap
  namespace: default
  resourceVersion: "667"
  uid: 7c42ef92-b87b-408e-bfc9-9d715e3b55e0
```

You should see the example-configmap with the keys and values specified.

Step 3: Create a ConfigMap from a File

Create a file named **config.txt** with the following content:

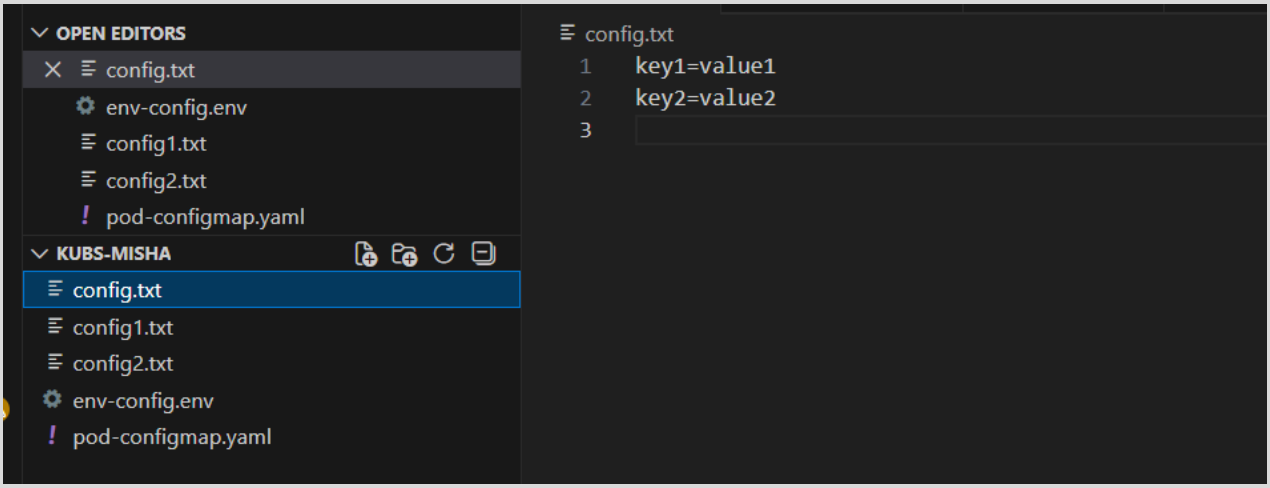
```
PS D:\kubs-misha> New-Item config.txt


    Directory: D:\kubs-misha


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        17-03-2026     13:01              0 config.txt
```

key1=value1

key2=value2



Create a ConfigMap named file-configmap from the file:

kubectl create configmap file-configmap --from-file=config.txt

```
PS D:\kubs-misha> code .
PS D:\kubs-misha> kubectl create configmap file-configmap --from-file=config.txt
configmap/file-configmap created
```

Verify the ConfigMap:

kubectl get configmap file-configmap -o yaml

```
PS D:\kubs-misha> kubectl get configmap file-configmap -o yaml
apiVersion: v1
data:
  config.txt: "key1=value1\r\nkey2=value2\r\n"
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-17T07:31:45Z"
  name: file-configmap
  namespace: default
  resourceVersion: "758"
  uid: 55604f06-b58a-45b6-8ab6-b10cace16c14
```

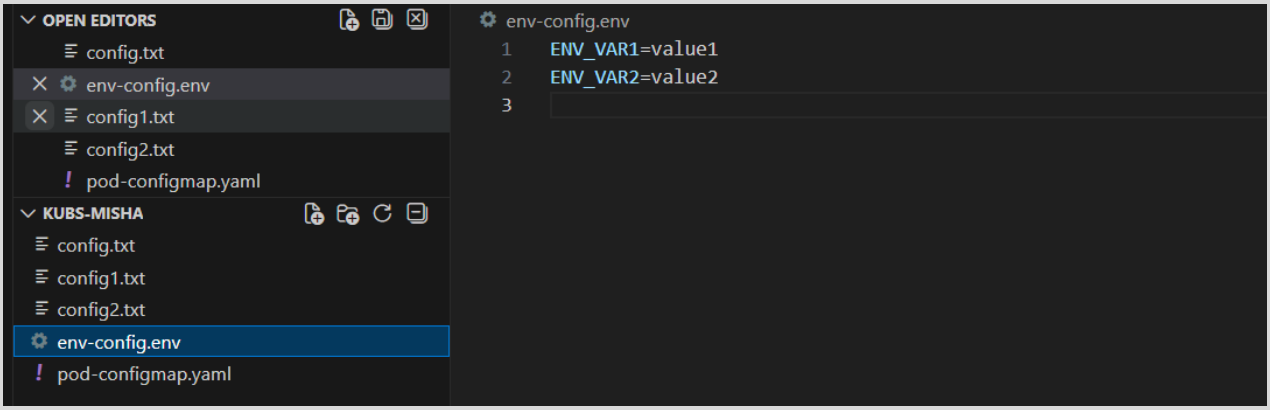
You should see the file-configmap with the content of config.txt.

Step 4: Create a ConfigMap from an Environment File

Create an environment file named **env-config.env** with the following content:

ENV_VAR1=value1

ENV_VAR2=value2



Create a ConfigMap named env-configmap from the environment file:

```
kubectl create configmap env-configmap --from-env-file=env-config.env
```

```
PS D:\kubs-misha> kubectl create configmap env-configmap --from-env-file=env-config.env
configmap/env-configmap created
```

Verify the ConfigMap:

```
kubectl get configmap env-configmap -o yaml
```

```
PS D:\kubs-misha> kubectl get configmap env-configmap -o yaml
apiVersion: v1
data:
  ENV_VAR1: value1
  ENV_VAR2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-17T07:32:40Z"
  name: env-configmap
  namespace: default
  resourceVersion: "832"
  uid: fef8a1a1-91ff-419f-b8d9-c04b9c253703
```

You should see the env-configmap with the environment variables specified in the file.

Step 5: Create a ConfigMap from Multiple Files

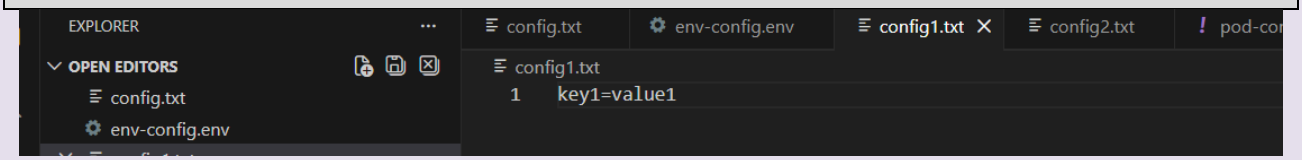
Create multiple files:

config1.txt with content:

```
key1=value1
```

config2.txt with content:

```
key2=value2
```



Create a ConfigMap named multi-file-configmap from multiple files:

```
kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
```

Verify the ConfigMap:

```
kubectl get configmap multi-file-configmap -o yaml
```

```
configmap/multi-file-configmap created
PS D:\kubs-misha> kubectl get configmap multi-file-configmap -o yaml
apiVersion: v1
data:
  config1.txt: key1=value1
  config2.txt: key2=value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-17T07:33:30Z"
  name: multi-file-configmap
  namespace: default
  resourceVersion: "904"
  uid: efe70b59-2e35-4a9e-90f5-c0da23d94d0e
PS D:\kubs-misha> kubectl apply -f pod-configmap.yaml
```

You should see the multi-file-configmap with the content of both config1.txt and config2.txt.

Step 6: Use ConfigMaps in a Pod

Create a file named **pod-configmap.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

In this manifest:

- The Pod reads environment variables from the env-configmap.
- The Pod mounts the file-configmap at /etc/config.

Apply the manifest to create the Pod:

kubectl apply -f pod-configmap.yaml

```
PS D:\kubs-misha> kubectl apply -f pod-configmap.yaml
pod/configmap-pod created
```

Check the status of the Pod:

kubectl get pods

```
PS D:\kubs-misha> kubectl get pods
NAME            READY   STATUS    RESTARTS   AGE
configmap-pod   1/1     Running   0          9s
```

View the Pod logs to see the environment variables and file content:

kubectl exec -it configmap-pod – printenv

```
PS D:\kubs-misha> kubectl exec -it configmap-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-pod
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
ENV_VAR1=value1
ENV_VAR2=value2
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
TERM=xterm
HOME=/root
```

You should see the environment variables and the content of the file-configmap.

Step 7: Clean Up

After completing the exercise, clean up the resources created:

```
kubectl delete pod configmap-pod
kubectl delete configmap example-configmap
kubectl delete configmap file-configmap
kubectl delete configmap env-configmap
kubectl delete configmap multi-file-configmap
rm config.txt env-config.env config1.txt config2.txt
```

```
PS D:\kubs-misha> kubectl delete pod configmap-pod
pod "configmap-pod" deleted from default namespace
PS D:\kubs-misha> kubectl delete configmap example-configmap
configmap "example-configmap" deleted from default namespace
PS D:\kubs-misha> kubectl delete configmap file-configmap
configmap "file-configmap" deleted from default namespace
PS D:\kubs-misha> kubectl delete configmap env-configmap
configmap "env-configmap" deleted from default namespace
PS D:\kubs-misha> kubectl delete configmap multi-file-configmap
configmap "multi-file-configmap" deleted from default namespace
PS D:\kubs-misha> rm config.txt env-config.env config1.txt config2.txt
```